

Technical Requirements Document for TDIS Portal

Author: Vidya Gummadi & Akhilesh Veerapareddy , Vijay Yerra, Murli Manickam

IDRT SME (Subject Matter Expert): Dr. Wesley Highfield

IDRT Data Analyst:

TDIS App Owner: Murali Manickam, Vidya Gummadi

Affiliations:

IDRT=Institute for a Disaster Resilient Texas; Texas A&M University

TDIS=Texas Disaster Information System; Texas A&M University

Table of Contents

Contents

Table of Contents	2
1 Introduction	5
1.1 Purpose	5
1.2 Intended Audience	5
1.3 Definitions, Acronyms, and Abbreviations	6
1.3.1 Scope	7
2 High-Level System Overview	8
2.1 System Context	8
2.2 Technical Architecture	8
2.2.1 Data Sources and Ingestion Entry Point (TDDL / Databricks)	9
2.2.2 TDDL Weather Ingestion Component (TDIS)	10
2.2.3 Transformation Layer (Silver) and PostgreSQL Sync	10
2.2.4 Events Sync Job	11
2.2.5 Technical Process	12
2.2.6 HSI Gold Layer Sync Job	12
2.2.7 Technical Process	12
2.2.8 Power Outage Sync Job	13
2.2.9 Technical Process	13
2.2.10 Shared Technical Behavior Across All Sync Jobs	14
2.2.11 Databricks Execution	14
2.2.12 Database Connection	14
2.2.13 Write Characteristics	14
2.2.14 Monitoring	14
2.2.15 Serving Layer: Flask API and Tile Server	14
2.2.16 Presentation Layer: React + Mapbox	15
2.2.17 CI/CD, Container Registry, and Operational Flow	16
2.3 Data Flows	16
2.3.1 Baron Data Flow (Wind, Hail, Rainfall)	16
2.3.1.1 Flow Summary	17

2.3.1.2 Detailed Flow	17
2.3.2 NIFC wildfire data flow	17
2.3.2.1 Flow Summary	18
2.3.2.2 Detailed Flow	18
2.3.2.3 Raw Landing (Bronze)	18
2.3.2.4 Texas and Status Filter	18
2.3.2.5 Normalize Wildfire Data to Unified Hazard Schema (Silver)	18
2.3.2.6 Write Filtered Output to Silver & Sync to PostgreSQL	19
2.3.3 USGS earthquake data flow	19
2.3.3.1 Flow Summary	19
2.3.3.2 Detailed Flow	19
2.3.3.3 Raw Landing (Bronze)	20
2.3.3.4 Texas Filtering	20
2.3.3.5 Normalize to Unified Hazard Schema (Silver)	20
2.3.3.6 Write Filtered Output to Silver & Sync to PostgreSQL	20
2.3.4 NOAA hurricane data flow	21
2.3.4.1 Flow Summary	21
2.3.4.2 Detailed Flow	21
2.3.4.3 Fetch Hurricane Advisories and Tracks from NOAA	21
2.3.4.4 Raw Landing (Bronze)	21
2.3.4.5 Texas / Gulf Region Filter	22
2.3.4.6 Normalize to Unified Hazard Schema (Silver)	22
2.3.4.7 Write Filtered Output to Silver & Sync to PostgreSQL	22
2.3.5 Precipitation HSI Data Flow	23
2.3.5.1 Fetch Precipitation Accumulation and Atlas 14 Data (Bronze)	23
2.3.5.2 Spatial Indexing (H3 L10)	23
2.3.5.3 Rolling Window Computation (Rainfall Maxima – Silver)	24
2.3.5.4 Texas Hex Grid Preparation (Static Mapping – Silver)	24
2.3.5.5 HSI Calculation (Gold Layer)	24
2.3.5.6 PostgreSQL Sync (Serving Layer)	25
2.3.6 Power Outage Data Flow	25
2.3.6.1 Flow Summary	25
2.3.6.2 Detailed Flow	25

2.3.6.3 Fetch Power Outage GeoJSON from Provider	25
2.3.6.4 Raw Landing (Bronze)	26
2.3.6.5 Texas Boundary GeoJSON Filter (Points and Polygons)	26
2.3.6.6 Administrative Join: State, County, City, Provider	26
2.3.6.7 Normalize to Unified Hazard Schema (Silver)	27
2.3.6.8 Aggregation: Statewide, County, City, Provider	27
2.3.6.9 Write Filtered Output and Aggregations to PostgreSQL.....	28
2.4 Database	29
2.5 Email API Backend.....	29
3 Security	29
3.1 Authentication and Authorization	29
3.2 Data Security	30
4 Scalability	30
5 Monitoring and Logging.....	30
5.1 Monitoring.....	30
5.2 Logging.....	30
6 Deployment	30
6.1 CI/CD Pipeline	30
6.2 Deployment Strategy	30
7 Backup and Disaster Recovery	31
7.1 Backup Strategy	31
7.2 Disaster Recovery Plan.....	31
8 Testing	31
8.1 Testing Strategy	31
8.2 Acceptance Criteria	32

1 Introduction

1.1 Purpose

The Texas Disaster Information System (TDIS) Portal is the public-facing web application used to surface disaster-related information for the State of Texas, with an initial focus on “Active Weather Events in Texas.” The portal aggregates data from multiple upstream providers, persists in curated event data in a centralized datastore, and exposes it through a modern web UI with geospatial visualizations.

This Technical Requirements Document (TRD) defines the reference architecture, integration patterns, data flows, and non-functional requirements for the modernized TDIS Portal implementation. It serves as the contract between product, engineering, and operations teams for what must be built, how it should behave, and how it will be operated in production.

The purpose of this document is to:

- Specify the end-to-end technical design of the TDIS Portal, including:
- Frontend React application hosted as an Azure Static Web App (SWA).
- Backend data access via Python Flask API and PostgreSQL.
- Scheduled databricks ingestion jobs for weather and hazard events.
- Supporting services such as email notifications and monitoring.
- Provide a single source of truth for implementation details that engineering teams will follow when designing, building, testing, and deploying the portal.
- Enable architecture and security review by documenting all external dependencies, runtime environments, and data-handling patterns.

This TRD is implementation-focused and is derived from the TDIS Portal PRD and system architecture documents maintained by IDRT/TDIS.

1.2 Intended Audience

This document is written for:

- **Frontend Engineers**
 - Build and maintain the React-based UI.
 - Integrate Mapbox for all geospatial layers.
 - Consume the **Python Flask REST API** hosted on Azure App Service.
- **Backend Engineers**
 - Implement and maintain the **Flask API layer**.
 - Define request/response contracts and schema guarantees.
 - Manage authentication, caching, and performance behavior for all API endpoints.
 - Interface directly with PostgreSQL.
- **Data Engineering Teams**

- Own and build all **Databricks ingestion pipelines**.
- Schedule, orchestrate, and monitor weather-event ingestion.
- Normalize raw external feeds into curated PostgreSQL tables consumed by the API.
- **DevOps / SRE**
 - Operate Azure App Service, Databricks jobs, and PostgreSQL.
 - Maintain observability, alerting, and deployment pipelines.
 - Manage Terraform/Terragrunt infrastructure and environment governance.
- **Product and Technical Leads**
 - Validate alignment with PRD and ensure requirement completeness.
 - Approve scope and verify architecture-level compliance.
- **Security and Compliance Reviewers**
 - Review data flows, API surface, authentication, and network boundaries.
 - Confirm compliance of the App Service + Databricks + PostgreSQL stack.

1.3 Definitions, Acronyms, and Abbreviations

- **TDIS**
Texas Disaster Information System; statewide platform delivering disaster-related data and public-facing applications.
- **TDIS Portal**
The public web application presenting curated hazard and weather data for Texas through a React UI and Mapbox visualizations.
- **IDRT**
Institute for a Disaster-Resilient Texas, Texas A&M University; program owner for the TDIS Portal.
- **Python Flask API / App Service API Layer**
The backend service replacing Cube.
Exposes REST endpoints for all event data queries.
Runs on **Azure App Service** as a stateless containerized or code-based deployment.
- **Databricks Weather Ingestion Pipeline**
Scheduled ingestion framework replacing all legacy batch jobs.
Pulls, standardizes, and writes hazard-event data into PostgreSQL.
Handles normalization, schema enforcement, and incremental updates.
- **PostgreSQL**
Primary datastore for curated hazard-event records.
Stores unified, query-optimized tables used by the Flask API.
- **Mapbox**
Geospatial rendering engine used by the front-end to visualize hazard layers, polygons, and markers.

- **Terraform / Terragrunt (IaC)**
Infrastructure-as-Code tooling used to provision, version, and operate all Azure resources for the portal.
- **Azure Static Web App (SWA)**
Hosting platform for the React application, providing global CDN distribution and static asset delivery.
- **Hazard Event**
A normalized data unit representing weather-related or environmental phenomena (wind, hail, rainfall, wildfire, earthquake, hurricane).
- **Provider Feed**
External authoritative data source (Baron Weather, NIFC, USGS, NOAA) ingested into the system via Databricks.

1.3.1 Scope

- **React Frontend (Azure Static Web App)**
 - Delivery of the public-facing TDIS Portal UI.
 - Mapbox-based visual layers for all supported hazard types.
- **Python Flask REST API (Azure App Service)**
 - Complete replacement for Cube.
 - All read-only endpoints for hazard-event retrieval.
 - Response shaping, pagination, filtering, and geospatial query handling.
- **Databricks Weather Ingestion Pipelines**
 - Extraction from external providers (Baron Weather, NIFC, USGS, NOAA).
 - Normalization and enforcement of the unified hazard-event schema.
 - Writes into PostgreSQL curated tables.
- **PostgreSQL Datastore**
 - Storage of all curated hazard-event data and related metadata.
 - Indexing and partitioning strategy enabling low-latency API access.
- **Email Submission Endpoint**
 - Flask API endpoint to process “Contact Us” submissions.
 - Email delivery to IDRT operational inboxes.
- **Infrastructure & Deployment**
 - Terraform/Terragrunt provisioning of App Service, Databricks, SWA, Postgres, Storage, and monitoring resources.
 - CI/CD pipelines for frontend, API, and ingestion code.
- **Monitoring & Observability**
 - Application Insights for API and SWA.
 - Databricks job telemetry.
 - Database health and capacity metrics.

Key Functional Areas Covered

- Weather-event retrieval across wind, hail, rainfall, wildfire, hurricane, and earthquake.
- Unified data model and deterministic ingestion.
- Map-based hazard visualization with geospatial overlays.
- High availability for public access.

2 High-Level System Overview

The TDIS Portal is a modular, cloud-native system built on Azure. It exposes curated hazard-event data through a React frontend backed by a Python Flask API and powered by Databricks ingestion pipelines writing into PostgreSQL. The system is designed for deterministic ingestion, low-latency retrieval, and fault-tolerant operation.

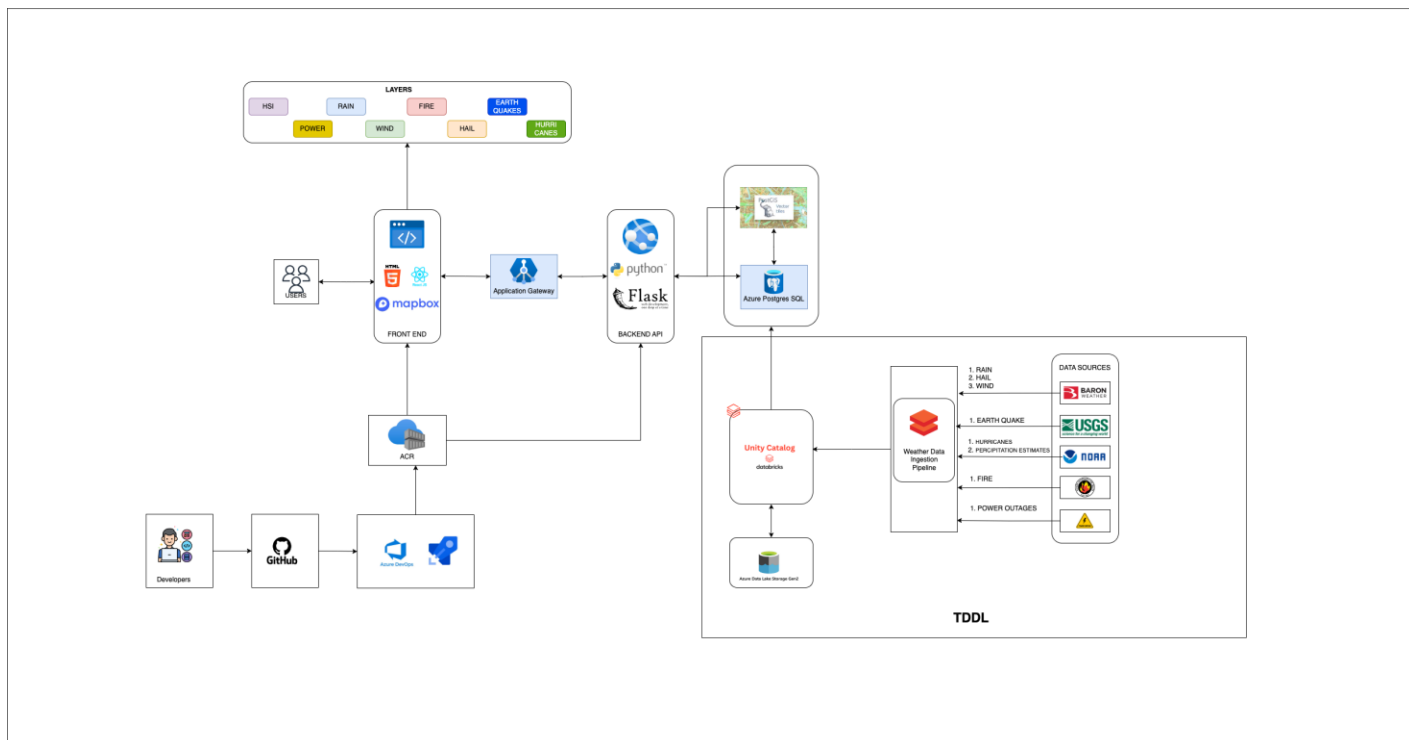
2.1 System Context

External Providers → Databricks Pipelines → PostgreSQL → Flask API → React UI

- External hazard sources supply raw weather and environmental feeds.
- Databricks pipelines extract, normalize, and load curated datasets into PostgreSQL.
- The Python Flask API reads from PostgreSQL and exposes the data via REST endpoints.
- The React frontend calls the API and visualizes hazards on Mapbox.
- Email submissions flow from the UI to the Flask API and are forwarded to operational inboxes.

2.2 Technical Architecture

The diagram below depicts the architecture for TDIS Portal application. This infrastructure is deployed using Terragrunt/Terraform.



2.2.1 Data Sources and Ingestion Entry Point (TDDL / Databricks)

External Data Providers

- **Baron Weather** – wind, hail, rainfall and other precipitation-derived products.
- **NIFC** – wildfire incidents and perimeters.
- **NOAA** – hurricanes and precipitation estimates.
- **USGS** – earthquake events.
- **Power Outage Feed** – utility outage polygons/regions.

All providers expose HTTP/HTTPS APIs or feeds that are pulled, never pushed.

Event	Source	API / Product	Filter Criteria
High Wind	Baron	Nationwide Damaging Wind Path (C39-0x03A1-0)	Exclude < 30
Hail	Baron	Severe Hail Path (C39-0x038A-0)	Exclude < 70%
Rainfall Accumulation	Baron	Rainfall Accumulation for 1 Hr (precip-liquidaccum-1hr)	Exclude < 0.1 inch
Wildfire	NIFC	WFIGS Fire History Locations	Exclude prescriptive fires and fires that started more than 24 hours ago
Earthquake	USGS	Earthquake Notifications, Feeds, and Web Services	None
Hurricane	NOAA	Current Storms	None

Power Outage	PO.us	Current Power Outages	Texas
--------------	-------	-----------------------	-------

2.2.2 TDDL Weather Ingestion Component (TDIS)

Refer to the **TDDL TRD document, Section 4**, for the process and technical details of the [Data Ingestion – Bronze Layer Strategy: Acquisition & Organization](#).

TDDL owns the “Weather Data Ingestion Pipeline” in Databricks.

- For each provider/hazard combination, a **Databricks Job** is configured with:
- Triggered interval (cron schedule)
- Source endpoint configuration (URL, auth, request template)
- Schema expectations and validation rules

Execution pattern per job:

- **Extract**
 - Call provider API according to schedule.
 - Capture raw JSON/CSV/GeoJSON payloads.
- **Persist Raw (Bronze)**
 - Write raw payloads to **Azure Data Lake Storage Gen2** in a provider-specific path structure:
 - `/bronze/{provider}/{hazard_type}/ingestion_date=YYYY-MM-DD/...`
 - Register or refresh **Unity Catalog** bronze tables pointing to these locations.
 - Raw tables provide full provider fidelity for replay and debugging.
- **Publish to Catalog (Bronze View)**
 - Expose bronze data through Unity Catalog so downstream jobs can consume it using standard SQL/Delta semantics.

2.2.3 Transformation Layer (Silver) and PostgreSQL Sync

Silver Transformation Pipelines

Refer to the **TDDL TRD document, Section 5**, for the process and technical details of the [5.Silver Layer Strategy – Standardization & H3 Indexing](#)

- Dedicated **Databricks “Silver” jobs** consume bronze tables and apply:
- Schema normalization into a unified hazard-event model.
- Geospatial standardization (e.g., GeoJSON/WKT, SRID normalization).
- De-duplication using provider keys and timestamps.
- Business rules (severity bucketing, status filtering, event lifecycle rules).
- Output is written as **Silver Delta tables** in Unity Catalog, typically partitioned by:
- `hazard_type`

- event_date or ingestion_date

PostgreSQL Sync Job

A separate **PostgreSQL sync job** runs in Databricks on a fixed schedule (or triggered after silver completes).

- Responsibilities:
 - Read from silver Delta tables.
 - Project into relational schemas optimized for API queries (fact tables + optional dimension tables).
 - Perform **idempotent upserts** into Azure Database for PostgreSQL using provider keys and composite business keys.
 - Maintain referential integrity and indexes required by the API (geospatial indexes, time indexes, hazard-type indexes).
- Sync pattern:
 - Incremental: only rows with updated_at > last_sync_timestamp are considered.
 - Atomic per batch: either a whole hazard-type batch commits or rolls back.
- Result:
 - PostgreSQL holds the **serving schema** used by the Flask API, while ADLS + Unity Catalog remain the system of record for historical and analytical workloads.

TDDL publishes data into PostgreSQL through **three dedicated sync jobs**, each responsible for a specific output category:

- **Silver Events Sync** – Publishes all hazard events (6 layers).
- **HSI Gold Sync** – Publishes rainfall HSI surfaces.
- **Power Outage Sync** – Publishes outage events.

The sync layer exists to make curated Delta tables available in PostgreSQL for **TDIS Portal**, **TDIS APIs**, and **external consumers** that require low-latency relational access.

2.2.4 Events Sync Job

Purpose: Push the unified Silver Events table into PostgreSQL.

Coverage:

- Rainfall
- Wind
- Hail
- Fire
- Earthquake
- Hurricane

2.2.5 Technical Process

- **Source**
 - Reads from the Silver Events table produced by the Silver pipeline.
 - Includes geometry (lon/lat), event metadata, severity/value, and provider attributes.
- **Transformation**
 - Standardizes column types to PostgreSQL-compatible formats.
 - Flattens JSON description fields if required.
 - Preserves event_id, event_type, and run timestamp for lineage.
- **Load Pattern**
 - **Upsert** or **truncate-and-insert** depending on the target schema.
 - Ensures PostgreSQL reflects the **latest Silver event state**.
 - Inserts all active events and deactivates or replaces older ones.
- **Usage**
 - Primary data source for TDIS Portal event maps.
 - Supports query patterns such as “latest hazards by type, region, or timestamp.”

2.2.6 HSI Gold Layer Sync Job

Purpose: Publish the Gold Rainfall HSI layer (hazard severity by H3).

Table Characteristics:

- Very large table (millions of hex records).
- Represents a **single snapshot in time**.
- Does not support incremental updates.

2.2.7 Technical Process

- **Source**
 - Reads the Gold HSI Delta table generated after rainfall maxima + Atlas14 blending.
- **Transformation**
 - Projects required fields:
 - H3 L10 hex ID
 - L6–L9 parent hex IDs
 - 3h/6h/12h/24h severity ratios (50-yr and 100-yr)

- Geometry (optional for Postgres)
- Run timestamp
- **Load Pattern**
 - **Full refresh (truncate → bulk insert).**
 - Required because HSI is always defined as a **fresh recomputed surface**, not an incremental dataset.
- **Usage**
 - Consumed by TDIS Portal and internal dashboards for hazard severity visualization.
 - Enables fast spatial lookups of rainfall severity.

2.2.8 Power Outage Sync Job

Purpose: Publish the processed outage events from the PowerOutage.us Bronze→Silver pipeline.

Coverage:

- Utility provider
- Coordinates
- Customers affected
- Outage cause
- Estimated restoration
- Record timestamp

2.2.9 Technical Process

- **Source**
 - Reads from the normalized outage dataset.
 - Includes lon/lat coordinates, outage identifiers, and provider metadata.
- **Transformation**
 - Maps Bronze/Silver schema to PostgreSQL table fields.
 - Ensures timestamp and numeric types conform to target schema.
 - Preserves job_id and fetched_at for lineage.
- **Load Pattern**
 - Typically **truncate-and-insert**, since the desired state is “current active outages only.”
 - If historical preservation is needed, the job switches to an **insert-only append** model.
- **Usage**
 - Power outage dashboards
 - TDIS Portal outage overlay layer
 - API consumers requiring current outage status

2.2.10 Shared Technical Behavior Across All Sync Jobs

Regardless of type, all three sync jobs follow the same low-level mechanics described in the document:

2.2.11 Databricks Execution

- Jobs run as Databricks notebook tasks or Python modules.
- Each job reads credentials from **Key Vault secret scopes**.
- Each job uses an **ephemeral job cluster** to perform the extraction + JDBC load.

2.2.12 Database Connection

Uses JDBC with:

- Host
- Database
- Username
- Password

Credentials are never stored in code; accessed via secret scopes.

2.2.13 Write Characteristics

- Parallel JDBC writes using multiple partitions for larger tables.
- Batch size tuned (typically 5k–20k rows).
- Transactionally safe pattern:
 - For full refresh tables: truncate in a controlled block, then bulk insert.
 - For append/upsert: staging + merge logic if needed.

2.2.14 Monitoring

- Each job logs:
- row_count
- run_status
- timestamps
- error messages

2.2.15 Serving Layer: Flask API and Tile Server

GitHub: <https://github.com/TexasDIS/tdis-portal-api>

Python Flask API (Backend)

- Deployed on **Azure App Service** as a stateless web application.
- Primary responsibilities:
 - Expose REST endpoints for hazard search, detail retrieval, and metadata.

- Execute SQL and geospatial queries against PostgreSQL.
- Enforce API contracts, validation, pagination, and error handling.
- Provide a **Contact-Us** endpoint that relays user messages to IDRT via SMTP or institutional email relay.

Access Patterns

Two main access modes from the frontend:

- **Direct JSON APIs**
 - For lists of events, detail cards, legends, metadata, and timestamps.
 - Endpoints like /hazards, /hazards/{id}, /hazards/metadata.
- **Tile Server Integration**
 - For high-density map layers (e.g., hail swaths, rainfall grids).
 - Flask exposes or proxies a tile service that returns vector or raster tiles (XYZ / Web Mercator).
 - The tile service uses PostgreSQL spatial tables or pre-rendered tiles generated from silver/postgres data.
 - All endpoints are stateless and can be scaled horizontally behind the App Service load balancer or Application Gateway.

2.2.16 Presentation Layer: React + Mapbox

GitHub: <https://github.com/TexasDIS/tdis-portal-frontend>

React Frontend

Single-page React application, hosted on Azure (Static Web App or containerized front-end behind Application Gateway).

- Responsibilities:
 - Render map and UI chrome (filters, legends, layer toggles).
 - Call Flask REST endpoints and tile endpoints.
 - Translate user interactions (filter selection, map pan/zoom) into API calls.

Mapbox Integration

- Mapbox GL renders:
 - Point layers (e.g., earthquakes, wind, hail).
 - Polygon layers (wildfire perimeters, power outage regions).
 - Line/track layers (hurricane tracks).
- Mapbox sources:
 - Vector/raster tiles from the tile server for large datasets.
 - GeoJSON feeds from direct API responses for smaller or detail views.

The frontend performs no heavy data transformation; it assumes payloads and tiles are already presentation ready.

2.2.17 CI/CD, Container Registry, and Operational Flow

Source Control

All code (React, Flask, Databricks notebooks/jobs definitions, Infra-as-Code) stored in **GitHub**.

Build and Release

Azure DevOps pipelines orchestrate:

- React build, tests, and deployment to Static Web App or front-end container.
- Flask build into a container image, tests, and push to **Azure Container Registry (ACR)**.
- App Service deployment from ACR.
- Databricks job deployment via REST/API/CLI (job definitions as code).
- Terraform/Terragrunt apply for infrastructure changes.

Runtime Topology

Users → Application Gateway (if present) → Frontend → Flask API → PostgreSQL.

Databricks jobs (TDDL ingestion + silver + Postgres sync) → ADLS Gen2 + Unity Catalog + PostgreSQL.

Observability

- Databricks: job runs, failures, and metrics visible via workspace UI and exported to central logging.
- Flask/API: monitored via Application Insights (latency, errors, dependency health).
- PostgreSQL: monitored for storage, connections, slow queries.
- Frontend: logs API errors and surfacing issues for operational debugging.

2.3 Data Flows

The following diagrams provide a high-level overview of data ingestion process based on various data sources.

2.3.1 Baron Data Flow (Wind, Hail, Rainfall)



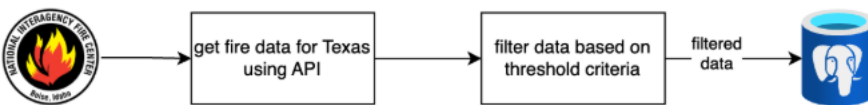
2.3.1.1 Flow Summary

Baron data is grid-based and product-specific. The pipeline queries each 1×1° grid cell covering Texas, retrieves product-level hazard values, filters to Texas-only records, normalizes geometry, applies severity logic, and publishes into the hazard-event model.

2.3.1.2 Detailed Flow

- **Extract Products per 1×1° Cell**
 - Pipeline divides Texas bounding box into 1×1 degree grid cells.
 - For each cell and each product (wind, hail, rainfall), call Baron endpoint.
- **Raw Landing (Bronze)**
 - Store raw product responses in ADLS Gen2 under /bronze/baron/<product>/YYYY/MM/DD/....
 - Register raw Delta tables in Unity Catalog.
- **Filter Out-of-Bounds Records**
 - Discard records where point or polygon geometry lies outside Texas geographic boundary.
- **Apply Threshold Criteria**
 - Apply hazard-specific business thresholds:
 - Wind speed threshold
 - Hail size threshold
 - Precipitation/rainfall thresholds
 - Remove values below significance level.
- **Normalize to Unified Hazard Schema (Silver)**
 - Convert geometry to GeoJSON/WKT.
 - Produce unified event structure: event_id, hazard_type, geometry, severity, etc.
- **Write Filtered Output to Silver & Sync to PostgreSQL**
 - Silver tables store cleaned, normalized records.
 - Postgres sync job performs idempotent upsert.

2.3.2 NIFC wildfire data flow



2.3.2.1 Flow Summary

NIFC provides national wildfire incident and perimeter data. The ingestion pipeline fetches the full NIFC dataset, lands raw payloads in ADLS Gen2 bronze, filters to Texas and relevant incident states, normalizes polygons and attributes into the unified hazard-event schema, and syncs the curated wildfire events into PostgreSQL for serving through the API.

2.3.2.2 Detailed Flow

- Databricks wildfire job runs on a configured schedule (e.g., every N minutes/hours).
- The job calls the NIFC API endpoints that expose incident and perimeter information.
 - Response includes:
 - Incident identifiers (e.g., incident_id, fire_name)
 - Status and lifecycle fields (active, contained, out)
 - Size metrics (acres burned, growth)
 - Geometries (polygons or multipolygons)
 - Timestamps (discovery, update, containment)

2.3.2.3 Raw Landing (Bronze)

- Store the raw NIFC payloads in **ADLS Gen2** under provider-specific paths:
/bronze/nifc/wildfire/YYYY/MM/DD/...
- Preserve provider schema as-is for traceability and replay.
- Register corresponding **bronze Delta tables** in Unity Catalog for downstream jobs.

2.3.2.4 Texas and Status Filter

Read bronze NIFC wildfire tables into Databricks.

- Filter incidents based on:
 - **Geospatial intersection** with the Texas boundary (polygon intersects or overlaps Texas).
- **Incident state** or business criteria:
 - Exclude closed/archived fires if they should not appear on the public map.
 - Optionally filter by minimum size (e.g., exclude extremely small fires if required).

2.3.2.5 Normalize Wildfire Data to Unified Hazard Schema (Silver)

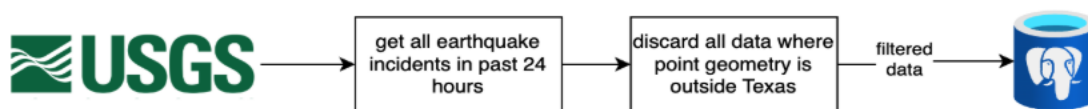
- Standardize geometry:
 - Convert perimeter polygons into a consistent GeoJSON/WKT format.
 - Ensure consistent SRID and geometry validity (fix invalid rings if necessary).
- Map NIFC fields into the unified hazard-event schema:
 - event_id – stable key derived from NIFC incident ID.
 - hazard_type – set to wildfire.
- geometry – wildfire perimeter or representative polygon.
- severity – derived from size, status, or configured rules.

- event_timestamp – primary incident timestamp (e.g., discovery or last update).
- ingestion_timestamp – Databricks ingestion time.
- provider – NIFC.
- attributes (JSON) – carry provider-specific fields such as incident type, percent containment, estimated costs, etc.
- Write normalized records into **Silver Delta tables** under:
/silver/hazards/wildfire/...
- Register or refresh **silver wildfire tables** in Unity Catalog.

2.3.2.6 Write Filtered Output to Silver & Sync to PostgreSQL

- Databricks **PostgreSQL sync job** reads from the silver wildfire tables.
- Transform into the serving schema:
 - Main hazard_events or wildfire_events table with all core fields.
 - Optional auxiliary tables for metadata or time-series if needed.
- Perform **idempotent upserts** into PostgreSQL using:
 - Key on event_id (and hazard_type = wildfire) as the primary business identifier.
- Ensure atomicity per batch:
 - Either an entire wildfire batch commit succeeds, or it is rolled back and retried.
- Result: PostgreSQL holds a clean, up-to-date set of wildfire incidents for Texas, ready for consumption by the Flask API and React/Mapbox UI.

2.3.3 USGS earthquake data flow



2.3.3.1 Flow Summary

USGS provides a continuous nationwide feed of recent earthquake events. The ingestion pipeline retrieves the latest 24-hour (or configured window) USGS dataset, stores raw responses in ADLS Gen2 bronze, filters events to those occurring within Texas, normalizes point-based seismic data into the unified hazard-event schema, and syncs the curated results into PostgreSQL for API consumption.

2.3.3.2 Detailed Flow

- Databricks earthquake ingestion job runs on a defined interval.
- The job calls the USGS “recent earthquakes” endpoint (e.g., last 24 hours or last hour).

- Payload includes:
 - event ID
 - latitude & longitude
 - depth
 - magnitude
 - location description
 - timestamps (event, updated)

2.3.3.3 Raw Landing (Bronze)

- Store the raw API response in **ADLS Gen2**:
/bronze/usgs/earthquake/YYYY/MM/DD/...
- Preserve provider formatting for replay and diagnostic use.
- Register **bronze Delta tables** in Unity Catalog.

2.3.3.4 Texas Filtering

- Load bronze USGS data into Databricks.
- Apply geospatial filter:
 - Only include earthquakes **within Texas boundaries**.
 - Use point-in-polygon spatial filtering via geospatial libraries (H3/Geospark/Databricks Geo functions).

2.3.3.5 Normalize to Unified Hazard Schema (Silver)

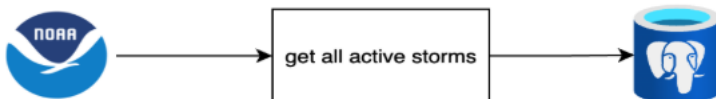
- Convert latitude/longitude into a standard geometry format (WKT/GeoJSON).
- Map USGS fields into the event schema:
 - event_id – USGS event unique ID
 - hazard_type – earthquake
 - geometry – point geometry
 - severity – derived from magnitude thresholds
 - event_timestamp – the USGS event time
 - ingestion_timestamp – Databricks job timestamp
 - provider – USGS
 - attributes – magnitude, depth, location name, and additional USGS metadata
- Write output to **Silver Delta tables** under:
/silver/hazards/earthquake/...
- Register/update the corresponding **silver earthquake table** in Unity Catalog.

2.3.3.6 Write Filtered Output to Silver & Sync to PostgreSQL

- Databricks **PostgreSQL sync job** reads the silver earthquake table.
- Transform into serving table format optimized for geospatial and time-range queries.
- Perform **idempotent upserts** using:
 - event_id

- hazard_type = earthquake
- Guarantee complete transactional integrity for each sync batch.
- PostgreSQL now contains ready-to-serve earthquake data for Flask API queries and React/Mapbox visualization.

2.3.4 NOAA hurricane data flow



2.3.4.1 Flow Summary

NOAA provides hurricane advisory, track, and intensity data for active tropical systems. The ingestion pipeline retrieves NOAA hurricane feeds, lands raw advisory packages in ADLS Gen2 bronze, filters to storms affecting Texas or the Gulf region, normalizes multi-point track geometry and intensity attributes into the unified hazard-event schema, and syncs curated hurricane events and track lines into PostgreSQL for API consumption.

2.3.4.2 Detailed Flow

2.3.4.3 Fetch Hurricane Advisories and Tracks from NOAA

- A scheduled Databricks hurricane job queries the NOAA APIs responsible for:
 - Active storm advisories
 - Forecast positions
 - Past track coordinates
 - Wind intensity metrics
 - Storm classification (TD, TS, hurricane category)
 - NOAA payloads typically include:
 - Advisory ID
 - Storm name and storm ID
 - Coordinate lists for historic & forecast track
 - Maximum sustained winds
 - Central pressure
 - Advisory timestamps
 - Cone polygons (if available)

2.3.4.4 Raw Landing (Bronze)

- Store raw JSON/XML payloads in **ADLS Gen2**:
/bronze/noaa/hurricane/YYYY/MM/DD/...

- Maintain original structure for debugging, audit, and replay.
- Register **bronze Delta tables** in Unity Catalog.

2.3.4.5 Texas / Gulf Region Filter

- Load bronze NOAA data into Databricks.
- Filter to storms relevant to Texas:
 - Storm track or cone polygon intersects Gulf of Mexico region.
 - Forecast path intersects or approaches Texas coastal or inland boundary.
 - Remove distant or irrelevant storms not impacting the region of interest.

2.3.4.6 Normalize to Unified Hazard Schema (Silver)

- Handle multi-geometry structure:
 - Convert track points into a **LineString** geometry.
 - Convert advisory area polygons (cones) into standard **Polygon** geometry.
 - Map NOAA fields into unified hazard-event structure:
 - event_id – unique advisory identifier or unique storm-time combination
 - hazard_type – hurricane
 - geometry – either track line or polygon (depending on representation)
 - severity – derived from storm category or wind intensity
 - event_timestamp – advisory timestamp
 - ingestion_timestamp – Databricks job timestamp
 - provider – NOAA
- attributes – structured JSON containing:
 - storm name
 - category
 - max sustained winds
 - pressure
 - movement direction and speed
 - forecast details
- Store outputs in **Silver Delta tables** in paths such as:
 - /silver/hazards/hurricane/track/...
 - /silver/hazards/hurricane/cone/...
- Register/update silver hurricane tables in Unity Catalog.

2.3.4.7 Write Filtered Output to Silver & Sync to PostgreSQL

- Databricks **PostgreSQL sync job** processes hurricane silver tables:
 - Sync **track geometry** as a LineString-based event series.
 - Sync **cone polygons** if required for visualization layers.
 - Sync advisory-level attributes into hazard_events-like serving schema.
- Perform **idempotent upserts** with composite keys such as:
 - event_id (advisory ID)

- hazard_type = hurricane
- Ensure atomic batch updates for accurate advisory progression.
- PostgreSQL now contains curated hurricane tracks and advisory geometries for Flask API use.

2.3.5 Precipitation HSI Data Flow

This section describes the full technical flow for the Precipitation Hazard Severity Index (HSI) from raw precipitation and Atlas 14 data through H3 indexing, rolling windows, HSI computation, and PostgreSQL sync.

For implementation details of the underlying medallion layers, refer to:

- TDDL TRD – [4.Data Ingestion – Bronze Layer Strategy: Acquisition & Organization](#) for Baron rainfall accumulation and NOAA Atlas 14 ingestion.
- TDDL TRD – [5.Silver Layer Strategy – Standardization & H3 Indexing](#) for rainfall sliding-window computation and H3 assignment.
- TDDL TRD – [6.Gold Layer Strategy – Business-Ready Products](#) for the HSI Gold layer data products and sync patterns.
- This document(**Transformation Layer (Silver) and PostgreSQL Sync**) – **PostgreSQL Sync** for the final sync of HSI into PostgreSQL.

2.3.5.1 Fetch Precipitation Accumulation and Atlas 14 Data (Bronze)

- Fetch **precipitation accumulation forecasts** for multiple point coordinates within Texas using the Baron rainfall accumulation APIs.
- Fetch **NOAA Atlas 14 precipitation intensity values** (e.g., 3h/6h/12h/24h design storms for 50-year / 100-year return periods) and land them in Bronze.
- Land all raw Baron and Atlas 14 payloads in ADLS Gen2 Bronze, preserving original structure for replay, audit, and debugging.
- Register Bronze Delta tables for Baron rainfall and Atlas 14 stations in Unity Catalog as part of the standard Bronze strategy.

Notebooks: Baron rainfall ingestion notebook(s); Atlas 14 ingestion notebook(s).

Jobs: Baron rainfall ingestion job; Atlas 14 ingestion job.

2.3.5.2 Spatial Indexing (H3 L10)

- Assign an **H3 Level-10 (L10) index** to every Baron precipitation forecast point.
- Assign an **H3 Level-10 (L10) index** to every NOAA Atlas 14 station.
- Persist H3 L10 indexes with each row so that all downstream joins between rainfall points, stations, and Texas hex grid can be done using H3 instead of expensive geometry operations.
- *Unity Catalog Tables:*
 - Bronze/Silver tables for Baron rainfall with h3_l10 column.
 - Bronze/Silver tables for Atlas 14 stations with h3_l10 column.

Notebooks: H3 indexing notebook for Baron and Atlas 14.

Jobs: H3 indexing job as part of the Silver standardization flow.

2.3.5.3 Rolling Window Computation (Rainfall Maxima – Silver)

- For each precipitation time series (per location), compute **maximum accumulated rainfall** over the following windows, measured from the run time:
 - 0–3 hours
 - 0–6 hours
 - 0–12 hours
 - 0–24 hours
 - Store, for each window:
- The **maximum accumulated value**.
 - The **start and end timestamps** of the window where this maximum occurs.
 - Persist these maxima in a Silver Delta table keyed by location/H3 and run timestamp.
- *Unity Catalog Table:*
tdis_data_catalog.weather_silver.precipitation_accumulation_windows

Notebook: Silver rainfall maxima notebook (rolling window computation).

Job: Rainfall Maxima Silver job.

2.3.5.4 Texas Hex Grid Preparation (Static Mapping – Silver)

- Tessellate the **Texas boundary** into a full **H3 L10 hex grid**.
- For each Texas H3 L10 hexagon, pre-compute and store:
 - The **nearest Baron precipitation forecast point** (for rainfall maxima linkage).
 - The **nearest NOAA Atlas 14 station** (for design storm thresholds).
- Parent H3 levels (L6–L9) and associated geometries for efficient aggregation and visualization.
- *Unity Catalog Table:*
tdis_data_catalog.weather_silver.TX_atlas14_pw_joined_h3_l10_static

Notebook: Texas hex grid and nearest-neighbor join notebook.

Job: Texas hex grid preparation job.

2.3.5.5 HSI Calculation (Gold Layer)

- For each **Texas H3 L10 hexagon**, compute Precipitation HSI by comparing observed rainfall maxima against Atlas 14 intensities:
 - Use the **closest precipitation window point** and **nearest Atlas 14 station**, as pre-computed in the Texas hex grid table.

- For each window (3h, 6h, 12h, 24h), compute:
 - $\text{HSI_window} = \text{observed_max_window} / \text{atlas14_window_intensity}$.
 - Compute **HSI values separately** for each of the 3h, 6h, 12h, and 24h windows.
 - For visualization and multi-scale analysis, compute **aggregated HSI** at parent H3 levels **L6 to L9**, taken as averages of child L10 hex HSI for each window.
- *Unity Catalog Table:*
tdis_data_catalog.weather_gold.precipitation_hsi

Notebook: HSI Gold layer notebook (HSI computation).

Job: Rainfall Maxima HSI Gold job.

2.3.5.6 PostgreSQL Sync (Serving Layer)

- Use the **HSI Gold layer sync** described in **TDDL Gold Layer Strategy – Business-Ready Products** and this document’s **PostgreSQL Sync** section to publish the precipitation_hsi table into PostgreSQL.
- Apply a **full refresh (truncate + bulk insert)** pattern so PostgreSQL always represents a single, consistent HSI surface for the latest run.
- Expose the HSI surface through TDIS Portal and APIs for map visualization and downstream hazard scoring.

2.3.6 Power Outage Data Flow

2.3.6.1 Flow Summary

Power outage data is delivered as geospatial features (points and polygons) representing outage regions, circuits, or assets. The ingestion pipeline retrieves GeoJSON feeds from the outage provider, lands raw payloads in ADLS Gen2 bronze, filters and clips features to Texas using a canonical Texas boundary, normalizes all geometries into the unified hazard-event schema, and syncs curated outage events into PostgreSQL. In addition, the pipeline computes **statewide, county-level, city-level, and provider-level outage aggregations** to support summary views in the portal.

2.3.6.2 Detailed Flow

2.3.6.3 Fetch Power Outage GeoJSON from Provider

- Scheduled Databricks **power outage job** calls the provider’s outage API.
- Provider returns **GeoJSON** (or equivalent) containing:
 - Point features (substations, centroid points, devices).

- Polygon/MultiPolygon features (outage regions, service areas).
- Attributes:
 - outage ID
 - utility/provider name
 - customers affected
 - estimated restoration time
 - outage status
 - timestamps (start, last updated)
- Entire payload is captured without modification.

2.3.6.4 Raw Landing (Bronze)

- Write raw GeoJSON payloads to **ADLS Gen2**:
/bronze/power_outage/YYYY/MM/DD/...
- Preserve full provider schema and geometry.
- Register **bronze Delta tables** in Unity Catalog that reference these bronze locations.

2.3.6.5 Texas Boundary GeoJSON Filter (Points and Polygons)

- Load bronze outage features into Databricks.
- Load canonical **Texas boundary GeoJSON** from a reference path or table, e.g.:
/ref/boundaries/texas_state_boundary.geojson.
- For each outage feature:
 - If geometry is a **Point**:
 - Run point-in-polygon against the Texas boundary.
 - Keep only points inside Texas.
 - If geometry is a **Polygon/MultiPolygon**:
 - Run polygon intersection with the Texas boundary.
 - Keep only polygons that intersect Texas.
 - Optionally clip the polygon to Texas for visualization and storage consistency.
 - Drop all features outside Texas.

2.3.6.6 Administrative Join: State, County, City, Provider

- Load reference boundary layers from catalog (Delta/Geo tables):
 - **State** (Texas single polygon – mostly used as a constant).
 - **County boundaries** with county FIPS and names.
 - **City boundaries** (incorporated areas) with city names and IDs.
- For each filtered outage geometry:
- Spatially join to:

- **County polygon** to derive county_id, county_name.
 - **City polygon** (where applicable) to derive city_id, city_name.
 - Retain provider information directly from outage attributes:
 - provider_id, provider_name.
- Result: each outage event is tagged with:
 - state = Texas
 - county_id, county_name
 - city_id, city_name (if resolvable)
 - provider_id, provider_name

2.3.6.7 Normalize to Unified Hazard Schema (Silver)

- Standardize geometry to consistent representation (GeoJSON/WKT + SRID).
- Map to unified hazard-event schema:
 - event_id – stable outage identifier.
 - hazard_type – power_outage.
 - geometry – point or polygon.
 - severity – derived from customers affected, duration, or business thresholds.
 - event_timestamp – outage start or last update time.
 - ingestion_timestamp – Databricks job run.
 - provider – outage provider / utility.
 - state, county_id, county_name, city_id, city_name.
 - attributes – JSON with full provider metadata.
- Write normalized outages to **Silver Delta tables**:
/silver/hazards/power_outage/events/...
- Register/refresh **silver power_outage events** table in Unity Catalog.

2.3.6.8 Aggregation: Statewide, County, City, Provider

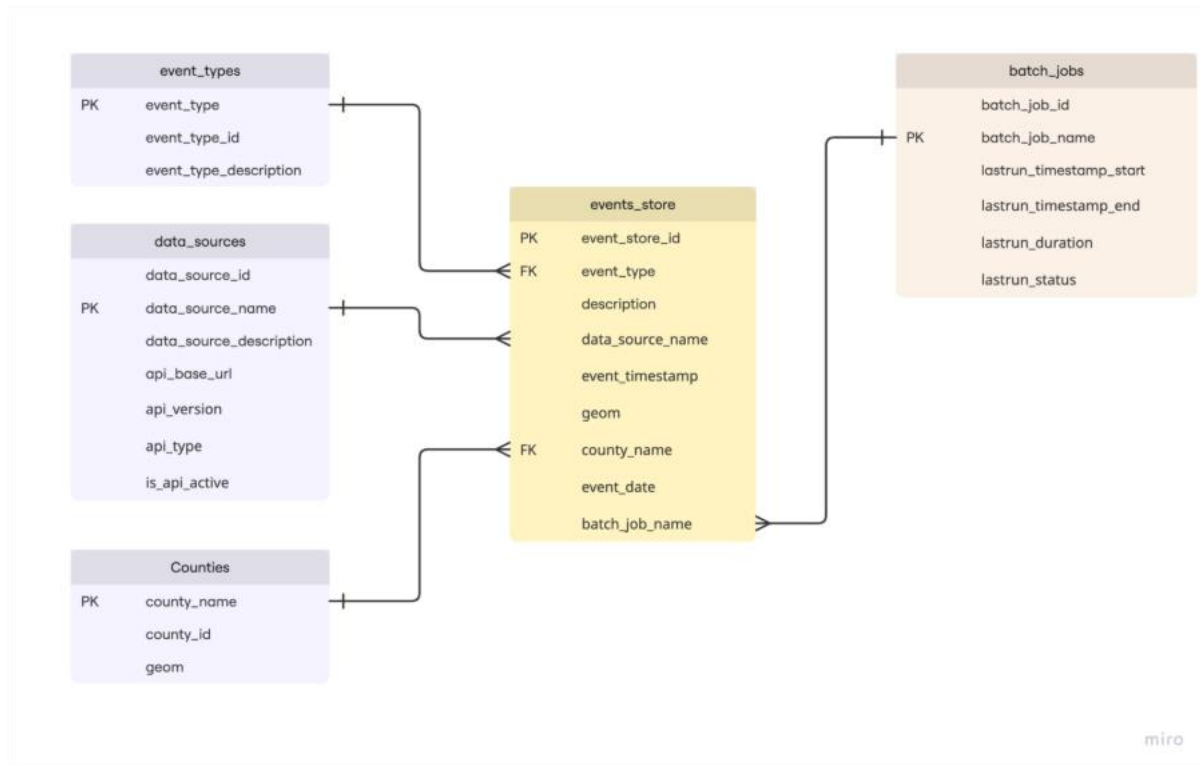
- From the silver outage events table, compute derived aggregations:
- **Statewide aggregation** (Texas):
 - Total customers affected.
 - Total number of active outage events.
 - Breakdown by severity buckets.
- **County-level aggregation**:
 - Group by county_id, county_name.
 - Metrics:
 - customers affected
 - outage count
 - earliest/median/latest outage start

- max severity
- **City-level aggregation:**
 - Group by city_id, city_name where available.
 - Same metric set as county-level.
- **Provider-level aggregation:**
- Group by provider_id, provider_name.
- Metrics:
 - customers affected
 - outage count
 - breakdown by county/city if needed as nested results.
- Store aggregations as separate silver tables, for example:
 - /silver/hazards/power_outage/agg_state/...
 - /silver/hazards/power_outage/agg_county/...
 - /silver/hazards/power_outage/agg_city/...
 - /silver/hazards/power_outage/agg_provider/...
- Register corresponding **aggregation tables** in Unity Catalog.

2.3.6.9 Write Filtered Output and Aggregations to PostgreSQL

- Databricks **PostgreSQL sync job** performs two types of sync:
- **Event-level sync**
 - Reads from silver power_outage events table.
 - Upserts into hazard_events or power_outage_events serving table in PostgreSQL.
- **Aggregation-level sync**
- Reads from aggregation tables:
 - state, county, city, provider.
- Upserts into dedicated serving tables:
 - power_outage_agg_state
 - power_outage_agg_county
 - power_outage_agg_city
 - power_outage_agg_provider
 - All sync operations are **idempotent** and **transactional** per batch.
- Result in PostgreSQL:
 - Event-level outages for detailed map rendering.
 - Statewide, county, city, and provider summary tables for:
 - dashboard tiles
 - summary panels
 - choropleth layers in the React/Mapbox UI.

2.4 Database



GitHub repo: <https://github.com/TexasDIS/tdis-portal-db> (DB migration scripts)

2.5 Email API Backend

- There are two scenarios where an email is generated and sent by the application:
 - When a user submits "Contact Us" form, an email is sent to tdis@tamu.edu
 - When user enters their email address in "Want to stay aware of TDIS activities?" section and clicks Subscribe button, an email is sent to idrt@tamu.edu
- In both these scenarios, the frontend calls APIs in the backed application to send the email.

GitHub repo: <https://github.com/TexasDIS/tdis-portal-backend/>

Note: In future, this backend application will be replaced by a common TDIS notification application that will be used to send these emails.

3 Security

3.1 Authentication and Authorization

The application will be publicly accessible. So there will not be any authentication.

3.2 Data Security

- Data Encryption: Data is not encrypted since there are no confidential information
- Network Security: SECURITY USED (Azure network security)

4 Scalability

- Horizontal Scaling: Azure App Service and ADLS can scale horizontally to accommodate an increasing number of users and larger flood model files.
- Auto-Scaling: Azure App Service and other compute resources are configured to auto-scale based on workload demands.

5 Monitoring and Logging

5.1 Monitoring

- Azure Monitor: Monitors the health and performance of Azure resources, including App Services, ADLS, and SQL Database.
- Application Insights: Details.

5.2 Logging

- Azure Log Analytics: Centralizes logs from all components, including web application actions, data uploads, and approval workflow steps.

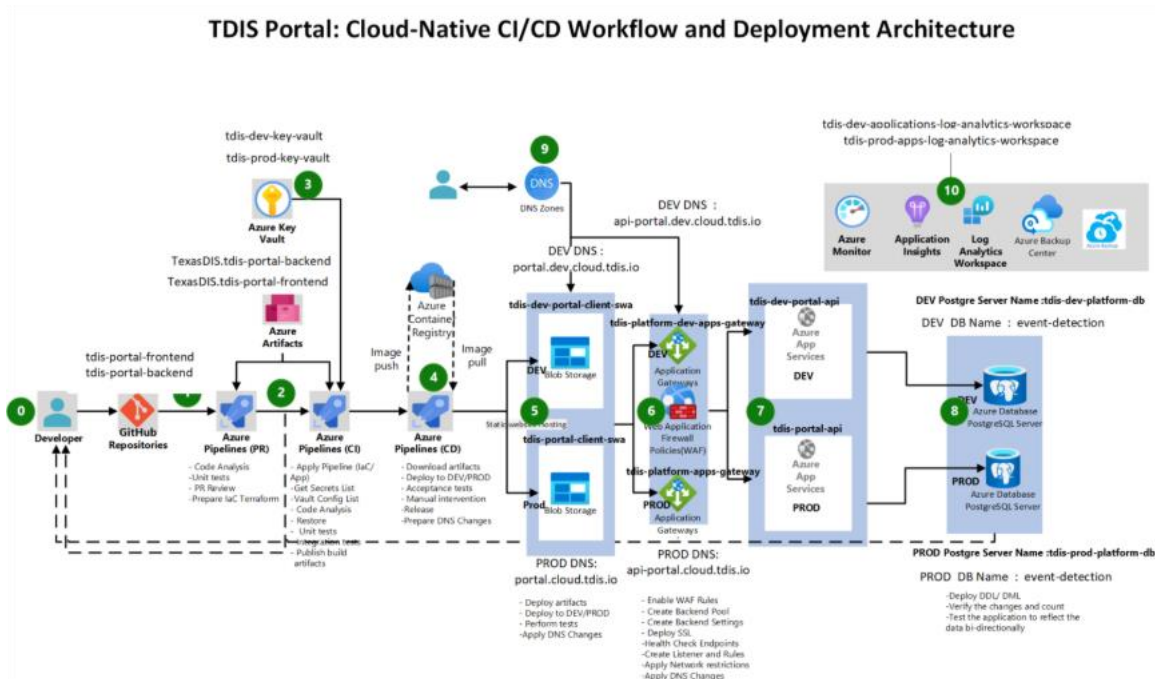
6 Deployment

6.1 CI/CD Pipeline

- Azure DevOps: Utilized for continuous integration and continuous deployment (CI/CD) of web applications and backend services.
- Infrastructure as Code (IaC): Managed using Azure Resource Manager (ARM) templates and terraform to ensure consistent and repeatable deployments.

6.2 Deployment Strategy

Blue-Green Deployment: Deployed in a manner that ensures zero downtime by running new versions of the application alongside the existing ones until the new version is validated.



<https://wikis.dev.cloud.tdis.io/en/tdis/portal-application/PortalAppDeploymentArchitecture>

7 Backup and Disaster Recovery

7.1 Backup Strategy

- **Automated Backups:** Regular automated backups are configured for Azure SQL Database and critical application configurations.
- **Retention Policies:** Backup data is retained according to the organization's data retention policies, ensuring recovery capability in case of data loss.

7.2 Disaster Recovery Plan

- **Geo-Redundancy:** Critical data in ADLS and SQL Database is geo-replicated to ensure availability in case of a regional outage.
- **Failover Procedures:** Automated failover procedures are in place for minimal disruption in case of a disaster.

8 Testing

8.1 Testing Strategy

- **Unit Testing:** Automated unit tests are executed via the CI/CD pipeline to validate individual components.
- **Integration Testing:** Ensures that the entire workflow, from metadata entry to model approval, functions as expected.

- **Performance Testing:** Load testing is conducted to ensure the application can handle expected user load and large file uploads.

8.2 Acceptance Criteria

- All services and components must pass automated and manual testing before deployment to production.
- The application must meet performance benchmarks and regulatory compliance standards.